CHAPTER 33

The Verdict

Chapter 1 asked a question that sounds simple: *is it true that Rust's ownership and lifetime concepts are based on affine types?* At the time, almost every word in it was undefined. We did not know what a type system is in the precise sense, what a structural rule is, what "affine" means, what a region is, or what it would mean for a programming language to be "based on" a logic. Thirty-two chapters later, each of those words has a definition, most of the definitions have Lean formalizations, and the key claims have been proved, tested, or both.

This chapter assembles the answer. It contains no new theory and almost no new code. Instead it goes back through the book, picks up the results that bear on the question, and puts them side by side. The answer, as Chapter 1 promised, is that the claim is **half true**, and the half that is true is not the half most people mean when they repeat it.

In this chapter you will learn:

- what the question means once each of its words has a precise meaning;
- which parts of Rust's discipline are affine, in the sense of a theorem we proved, and with which amendments;
- why borrowing and lifetimes are a different theory, and how much of the aliasing rule affinity does explain;
- the three senses of "based on", and the answer in each;
- the verdict in tables: Rust features, structural rules and error codes;
- how to say it correctly, and what the two languages of this book taught us about each other.

33.1 The question, made precise

The claim has two parts, ownership and lifetimes, and one predicate, "based on affine types". We now know what each of them means.

Affine. A logic or type system is *affine* when it keeps the structural rule of weakening and drops the rule of contraction (Chapters 6 and 21). Read bottom-up, weakening says an assumption may go unused, and contraction says an assumption may be used more than once. So in an affine system every assumption is used *at most once*. A *linear* system drops weakening too, so every assumption is used *exactly once*. The affine λ-calculus λ^{aff} of Chapter 22 is the type-theoretic version: its typing rules

split the context between subterms that both run, and its central metatheorem, the usage theorem, says that in a well-typed term every variable occurs at most once.

Ownership. By "ownership" people mean the discipline of moves: a value of a non-`Copy` type has one owner, passing it to a function or binding it to another variable moves it, and the old name cannot be used again. Around this core are `Copy` (types whose values are duplicated implicitly), `Clone` (explicit duplication) and `Drop` (what happens when a value is discarded).

Lifetimes. By "lifetimes" people mean the annotations `'a` on references, together with everything the borrow checker enforces about references: that they do not outlive their referents, and that a value is not mutated while it is shared.

Based on. This is the slipperiest word, and Chapter 1 split it into three senses:

1. *Historical*: the designers took the theory of affine types as an input.
2. *Specificational*: the rules the compiler enforces are the rules of an affine type system.
3. *Implementational*: the compiler works by building derivations in an affine type system.

The rest of the chapter evaluates the two parts of the claim in these senses.

33.2 Ownership: affine, with three amendments

The core is a theorem

Here is the program that opened Part V's story, from Chapter 6:

```
pub fn checkout(coupon: Coupon) -> u32 {
    let first = redeem(coupon);
    let second = redeem(coupon);
    first + second
}
```

```
error[E0382]: use of moved value: `coupon`
  --> compile_fail/ch06_use_after_move.rs:16:25
   |
14 | pub fn checkout(coupon: Coupon) -> u32 {
   |                 ------ move occurs because `coupon` has type `Coupon`, which does not
implement the `Copy` trait
15 |     let first = redeem(coupon);
   |                         ------ value moved here
16 |     let second = redeem(coupon);
   |                          ^^^^^^ value used here after move
   |
note: consider changing this parameter type in function `redeem` to borrow instead if owning
the value isn't necessary
  --> compile_fail/ch06_use_after_move.rs:9:23
   |
 9 | pub fn redeem(coupon: Coupon) -> u32 {
   |        ------         ^^^^^^ this parameter takes ownership of the value
   |        |
   |        in this function
note: if `Coupon` implemented `Clone`, you could clone the value
  --> compile_fail/ch06_use_after_move.rs:5:1
   |
 5 | pub struct Coupon {
   | ^^^^^^^^^^^^^^^^^ consider implementing `Clone` for this type
...
15 |     let first = redeem(coupon);
   |                         ------ you could clone this value
```

In the language of Chapter 22, the body of `checkout` is an application of `redeem` to `coupon` paired with another application of `redeem` to `coupon`. The pairing rule splits the context: each variable of the context goes to the left premise or to the right premise, never to both, unless its type is `Copy`. There is no way to split $coupon : \mathit{Coupon}$ so that both premises receive it, so there is no derivation. Error E0382 is rustc's report of the failed split, and the compiler's notes even name the two ways out that the theory offers: borrow instead of consuming (Part VI), or make the type duplicable with `Clone` (Chapter 23).

This is not a loose analogy. For λ^{aff} we proved in Lean the **usage theorem**: in every well-typed term, every variable is used at most once, counting the two alternatives of an offer or a branch with a maximum rather than a sum. Its proof rests on a single lemma about context splitting, and the algorithmic checker that threads "leftover" contexts was proved sound and complete with respect to the typing rules. The `affine` crate implements the same algorithm for a Rust-like syntax and reproduces the core of rustc's E0382 messages. The move checker on the non-`Copy` fragment of Rust enforces exactly the invariant of this theorem. In the *specificational* sense, this part of the claim is true.

It needs three amendments and one refinement, each of which the book worked out.

Amendment 1: `Copy` restores contraction

Not every type in Rust is affine. For `Copy` types, contraction and weakening are silent: `let b = a;` copies, and `a` remains usable. Chapter 23 identified this with linear logic's exponential modality: a `Copy` type behaves as if it had the form $!\tau$, for which the structural rules hold again. The side conditions of `Copy` are the side conditions of the exponential. *Promotion* says that a duplicable value can only be built from duplicable parts, and rustc enforces it as E0204 (a struct with a `String` field cannot be `Copy`). A value with a destructor cannot be duplicated silently, which is E0184.

Formally, λ^{aff} with `Copy` adds one splitting rule, S-Both, which sends a `Copy` variable to both premises, and leaves every typing rule unchanged. The generalized usage theorem, proved in Lean, says that non-`Copy` variables are still used at most once. Chapter 23 also proved why promotion is not a technicality: if function types were `Copy` regardless of what they capture, a well-typed program could step to an ill-typed one. So the accurate statement is that Rust is affine *for non-`Copy` types*, and structural for `Copy` types.

Amendment 2: `Clone` is a term, not a rule

`Clone` is often mentioned together with `Copy`, but in the theory it is something else entirely. A structural rule is invisible: it changes what the context may do without appearing in the term. `Clone` is visible: `x.clone()` is an ordinary function call, a term of type $\tau \multimap \tau \otimes \tau$ that consumes a borrow of its input and produces a new value, at a cost and possibly with effects. Every affine system can contain such terms for particular types; having them does not make the system less affine. `Rc` and `Arc` go one step further and make sharing itself a counted resource: each clone and each drop of the pointer is bookkept, which is precisely why they cannot be `Copy`.

Amendment 3: weakening has an effect, and nothing forces consumption

Weakening is kept: you may let a value go out of scope unused. But in Rust, discarding is not free. It runs the value's destructor (Chapter 24). The typing rule is affine, but the dynamics are not indifferent to the discard. The book's slogan was *affine in typing, linear in dynamics*: every value that is not moved is dropped exactly once, at a point fixed by the rules of drop order.

Then came the second half of that chapter. "Exactly once" is a property of the ordinary dynamics, not a guarantee of the type system: `mem::forget`, `Box::leak` and reference cycles of `Rc` can all prevent a destructor from running, and all of them are safe. Since 2015, Rust's soundness may not depend on destructors running. For the small resource language of Chapter 24 we proved in Lean that no resource is ever released twice, that without `forget` every resource is released exactly once, and that `forget` breaks the second property but not the first. That is the precise difference between affine and linear. Rust guarantees "at most once"; nothing in Rust 1.99 lets you declare a type whose values *must* be consumed. That is why typestate (Chapter 25) can prevent a protocol from being used wrongly but cannot force it to be finished.

The refinement: places, not variables

λ^{aff} counts uses of *variables*. Rust counts moves of *places*: you may move one field out of a struct and keep using the others, and the compiler tracks this per field (Chapter 22). This is a refinement of the same discipline, not a different one: a pair in λ^{aff} is taken apart by a `let` that binds both components at once, and Rust's partial moves are what you get when the pattern is spread out over several statements and the compiler keeps track of which parts are gone.

So the first half of the claim, stated with care, is true: **Rust's ownership discipline is an affine type system on non-`Copy` types, with `Copy` playing the role of the exponential, `Clone` an explicit duplication term, and weakening attached to a destructor.**

33.3 Borrowing: a different theory

Shared references are not affine at all

Now the second half. Start with the simplest fact from Chapter 26:

```
/// Contraction, available only for `Copy` types: the value is used twice.
fn duplicate<T: Copy>(x: T) -> (T, T) {
    (x, x)
}

fn references_are_copy() {
    let report = String::from("sensor report");
    // `String` is not `Copy`, but `&String` is: a shared reference can be
    // duplicated freely, and both copies point to the same string.
    let (r1, r2) = duplicate(&report);
    println!(
        "r1 = {r1:?}, r2 = {r2:?}, same address: {}",
        std::ptr::eq(r1, r2)
    );
    // `duplicate(&mut report)` would not compile: `&mut String` is not `Copy`.
}
```

```
r1 = "sensor report", r2 = "sensor report", same address: true
```

`&T` is `Copy`, for every `T`. A shared reference can be duplicated, discarded and used in any order: it is a *fully structural* value, the exact opposite of an affine one. Whatever rejects the bad programs of Part VI, it cannot be a restriction on how many times a shared reference is used, because there is none.

Mutable references are different: `&mut T` is not `Copy`, and moving one is E0382, word for word as for an owned value. But look at the error that really characterizes mutable references:

```
pub fn transfer(accounts: &mut Vec<i64>, amount: i64) {
    let from = &mut accounts[0];
    let to = &mut accounts[1];
    *from -= amount;
    *to += amount;
}
```

```
error[E0499]: cannot borrow `*accounts` as mutable more than once at a time
 --> compile_fail/ch26_two_mut.rs:8:19
  |
7 |     let from = &mut accounts[0];
  |                ---------- first mutable borrow occurs here
8 |     let to = &mut accounts[1];
  |                   ^^^^^^^^ second mutable borrow occurs here
9 |     *from -= amount;
  |     ---------------- first borrow later used here
  |
  = help: use `.split_at_mut(position)` to obtain two mutable non-overlapping sub-slices
```

`from` and `to` are two *different* variables, and each is used exactly once. An affine checker counts uses of variables, so it would accept this program. The conflict is between two values that point into the same place, and a type system that only counts variables cannot see what they point to.

The dangling-reference term

Chapter 26 made the same point formally. Take λ^{aff}, add a heap with owned boxes, and add a duplicable reference type whose borrowing rule is "affine-respectable" (it consumes nothing). Every rule respects affinity, and yet one can type a term that frees a box and then reads it through a reference. Affine typing plus duplicable references is unsound. There are two ways to repair it, and Rust uses both:

- **lend and return**: borrowing consumes the owner for the duration of a delimited scope and gives it back afterwards;
- **regions**: every reference type carries the name of a region of the program, and no reference may be used outside its region.

Neither is about multiplicity. Both are about *time*: during which part of the program a reference is valid. That was the box in Chapter 26 that this whole chapter turns on: affinity is a statement about how many times; lifetimes are a statement about when.

How much of the aliasing rule is affinity?

Here the book's answer is more precise than the folklore, in both directions. Chapter 28 built λ^{aff}_{rgn}, a calculus with moves, drops, `Copy`, shared and mutable borrows, lifetimes and reborrowing, in which a

borrow *lends* its owner and returns it, so that a borrow expression has type $Box\,\tau \otimes \sigma$. In that calculus the classic aliasing bug

```
pub fn aliasing_bug() {
    let mut v: Vec<i32> = vec![1, 2, 3];
    let first: &i32 = &v[0];
    v.push(4);
    println!("{first}");
}
```

```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
  --> compile_fail/ch28_aliasing_bug.rs:9:5
   |
 8 |     let first: &i32 = &v[0];
   |                       - immutable borrow occurs here
 9 |     v.push(4);
   |     ^^^^^^^^^ mutable borrow occurs here
10 |     println!("{first}");
   |                ----- immutable borrow later used here
```

fails for an affine reason: the owner `v` would have to go to both premises of the borrow rule, once to the borrow and once to the code that pushes, and a `Box` is not `Copy`. Rustc says E0502; the calculus says "failed split". So affinity *does* contribute to the aliasing rule, more than the slogan "lifetimes are not affine" suggests.

But notice what made that possible. The split only works because the borrow is a *scoped construct* that holds the owner for the duration of its body: that is a region. And the calculus needs a second ingredient that no counting argument can supply, the **escape check**: a reference must not leave the scope of the borrow that created it, which is a condition on which region names may appear in a type ($\alpha \notin \mathrm{flv}(\sigma)$). Chapter 28 proved the usage theorem for the calculus (a mutable reference, like any non-`Copy` variable, is used at most once) and noted explicitly that the escape check is the part of the system the usage theorem cannot explain. So the aliasing discipline is affinity of owners *inside* a region discipline. Without the regions, affinity alone produces the unsound dangling-reference term.

Lifetimes are regions; their order is subtyping

What, then, is a lifetime? Chapter 26 answered: a *region variable*, in the tradition of Tofte and Talpin's region calculus and Cyclone. Functions are polymorphic over lifetimes; `'a: 'b` is an outlives constraint that orders them; `'static` is the largest region; elision fills in the obvious cases; the escape check (E0597, E0515) keeps references inside their regions. Chapter 27 added the rest: the outlives order induces Rust's only form of subtyping (`&'long T <: &'short T`), variance says how that subtyping lifts through type constructors, and variance turned out to be functoriality on the preorder category of lifetimes. The classic unsound program that invariance of `&mut T` prevents has nothing to do with how many times anything is used.

Permissions

Finally, Chapter 26 gave references a reading that unifies the rest: a reference is a *permission*. The owner has full permission (read, write, free); `&mut T` is a unique read-write permission; `&T` is a duplicable read permission; a borrow is a temporary transfer of permission from the owner to the reference, and the aliasing rule is a disjointness condition on permissions. That is the vocabulary of

fractional permissions and of separation logic, and Chapter 32 showed that the semantic proof of Rust's soundness, RustBelt, is written exactly in that logic. Permissions can be split and recombined; they are not counted as uses of a variable.

So the second half of the claim is false as stated. **Lifetimes are region variables, the outlives relation is subtyping, and the borrow rules are a permission discipline. Affinity contributes one ingredient (owners and mutable references cannot be duplicated), but the theory of borrowing is the theory of regions and permissions.**

33.4 How the compiler really does it

There remains the third sense of "based on". Does rustc *work* by building derivations in an affine type system, the way our `affine` and `borrowck` crates do?

It does not. Chapter 29 followed a program through the compiler: type checking with lifetimes as inference variables, lowering to MIR (a control-flow graph of basic blocks acting on places, with explicit storage markers and drops), and then a family of *dataflow analyses* over that graph. Moves are checked by computing, at every program point, which places may be uninitialized (E0382, E0381). Borrows are checked by computing regions as sets of program points from liveness and outlives constraints, then reporting every access that conflicts with a loan in scope (E0499, E0502, E0505, E0506, E0597). Two-phase borrows accept `v.push(v.len())`; the location-insensitive constraints of NLL still reject "problem case #3", which Polonius, a reformulation of the analysis in terms of loans and origins, accepts. The `mini-nll` crate reproduced rustc's error codes on real programs with these analyses, and in Lean we proved that the iterative solver computes the least fixpoint and that NLL regions are contained in lexical ones.

None of this is proof search. The relationship between the type systems of Parts V and VI and the compiler is the relationship between a *specification* and an *implementation*: the calculus says which programs are acceptable (and is in fact stricter than today's compiler, which accepts programs that lexically scoped borrows reject); the analysis decides acceptability efficiently, on a representation the calculus does not even have. In the implementational sense, the claim is false for both halves.

33.5 Three senses, three answers

Sense of "based on"	Ownership (moves, `Copy`, `Drop`)	Lifetimes and borrowing
Historical	plausible, but says little	regions, not affinity
Specificational	**true**: an affine system on non-`Copy` types	**false**: regions, subtyping, permissions
Implementational	false: dataflow on MIR	false: dataflow on MIR

The historical row deserves a word of caution, because it is the one this book can say least about. Rust's design drew on several research traditions, among them region-based memory management (Cyclone and the ML Kit), unique pointers and move semantics, and typestate, which early Rust had as a language feature and later removed. Affine and linear types were part of the same landscape. But

a historical influence is not a description: knowing that the designers read about affine types tells you nothing about which rules the compiler enforces. The specificational row is the one that answers the question a programmer actually cares about, and it is the one we could settle with proofs.

33.6 The verdict in tables

Rust features and their accounts

Rust feature	Correct account	Chapters
move of a non-`Copy` value	affine typing: contraction absent	6, 22
unused value, end of scope	weakening, kept, with a destructor as effect	6, 24
`Copy`	the exponential !: contraction and weakening restored	23
`Clone`	an explicit duplication term $\tau \multimap \tau \otimes \tau$	23
`Rc`, `Arc`	counted sharing of a heap value	23, 24
`mem::forget`, leaks	affine, not linear: no "must consume"	24
partial moves	the affine discipline, tracked per place	22
typestate	affinity makes old states unusable	25
`&'a T`	region-indexed duplicable read permission	26, 28
`&'a mut T`	region-indexed unique write permission (affine)	26, 28
`'a: 'b`, variance	outlives preorder, subtyping, functoriality	26, 27
reborrowing	lend-and-return of a reference within a nested region	28
the escape check	region discipline, not counting	26, 28
borrow checking in practice	dataflow over MIR (NLL; Polonius in development)	29
soundness of `unsafe` libraries	semantic typing in an affine separation logic	32

Which structural rules Rust keeps

	Exchange	Weakening	Contraction
owned non-`Copy` values	yes	yes, with `Drop`	no
`Copy` values	yes	yes	yes
`&'a mut T`	yes	yes	no
`&'a T`	yes	yes	yes

Only the first and third rows are affine. Shared references are in the fully structural row, together with integers.

Error codes and the theory behind them

Error	Meaning	Theory
E0382	use of moved value	no contraction (failed context split)
E0204, E0184	invalid `Copy`	side conditions of ! (promotion, no destructor)
E0499	two `&mut` to one place	uniqueness of write permission
E0502	`&mut` while shared	owner lent to a scoped borrow; permissions
E0505, E0506	move or assign while borrowed	the owner is lent for a region
E0597, E0515	reference outlives referent	escape check: regions
E0040, E0509	calling `drop`, moving out of `Drop`	"dropped at most once"
E0381	use of uninitialized	dataflow: maybe-uninitialized places

Of the borrow-related rows, only E0499 can be explained by counting uses of something, and even then the something is a permission to a place, not a variable.

33.7 Where affinity reappears

There is one more place where "affine" is exactly the right word, and it may be the origin of the folklore. Chapter 32 described how RustBelt proved Rust's type system sound: not by a syntactic argument like Chapter 9's, but by interpreting every type as a predicate in Iris, a higher-order concurrent separation logic, and proving that every well-typed program, and every library that uses `unsafe` correctly, satisfies its predicate. Iris is an **affine** separation logic: from $P * Q$ you may conclude P, discarding Q. We proved the corresponding fact for the separation logic of Chapter 32, and saw that it holds in the up-closed model and fails in the exact one.

So affinity is a correct description of the *logic in which Rust's soundness is proved*, one level below the language. It is not a description of the surface rules for borrowing. When someone says "Rust is based on affine logic", they may be compressing two true statements, "ownership is an affine type system" and "Rust's soundness proof lives in an affine separation logic", into one false one, "lifetimes are affine types".

33.8 How to say it correctly

> **IMPORTANT**
>
> **Say:** "Rust's ownership and move semantics implement an affine discipline: a binding of a non-`Copy` type may be used at most once, which is the absence of contraction. `Copy` restores contraction for a class of types, playing the role of linear logic's exponential, and `Clone` is explicit duplication. Weakening is kept, so values may be discarded (with a destructor running), which makes Rust affine rather than linear: it cannot force a value to be consumed. Borrowing and lifetimes are a different mechanism: a region system with an outlives order, variance and an escape check, and a permission discipline in which shared references are freely duplicable and mutable references are unique. The compiler checks all of it with dataflow analyses on MIR."
>
> **Do not say:** "Lifetimes are affine types." A shared reference is `Copy`, the opposite of affine; lifetimes measure extent, not multiplicity.

If you have only one sentence: *Rust's ownership is affine; its borrowing is regions and permissions.*

33.9 Two languages, one idea, revisited

Chapter 1 promised that Rust and Lean would illuminate each other. It is worth looking back at what each of them taught.

Rust's type system is a federation of theories. Its algebraic data types are the products and sums of Chapter 11. Its generics are System F restricted to rank one and implemented by monomorphization (Chapter 12), with local Hindley–Milner inference (Chapter 13). Its traits are type classes elaborated into dictionaries (Chapter 14), and `impl Trait` and `dyn Trait` are existential types (Chapter 15). Its functions are the morphisms of a category that is *not* cartesian closed for non-`Copy` types, because the diagonal needs `Clone` (Chapter 19). Its `?` is the bind of two monads it cannot name (Chapter 20). Its ownership is an affine type system (Part V), its borrowing a region and permission system (Part VI), its const generics a first step toward dependent types (Chapter 30), and its `unsafe` a door into semantic typing (Chapter 32). Each theory was invented separately, decades apart, and Rust's achievement is to make them work together in a practical language. Each also has a seam where Rust departs from the textbook, and those seams (`panic!` inhabiting every type, inference stopping at signatures, no higher-kinded types, leaks being safe, borrow checking by analysis) were among the most instructive parts of the book.

Lean's type theory is one theory. Everything we built in Lean, from the judgments of Chapter 3 to the separation logic of Chapter 32, was built from the same few ingredients: dependent function types, inductive types, a hierarchy of universes, and the impredicative universe `Prop`. A typing judgment was an inductive predicate; a derivation was a value of it; a metatheorem was a function on derivations; a category was a structure whose fields include proofs; a protocol was an indexed family; "this program is correct" was a type. Curry–Howard was not a slogan there but the way the system works.

Rust showed us where the ideas *live*: in error codes, in API design, in the compromises a language makes to be fast and usable. Lean showed us what the ideas *are*: definitions precise enough to be checked by a machine, and theorems that say exactly what a rule guarantees. Rust implements; Lean proves. Neither would have been enough alone.

33.10 Where to go next

Appendix F lists books and papers for every part of the book. A few directions stand out.

- **Lean and Mathlib.** *Theorem Proving in Lean 4*, *Functional Programming in Lean* and *Mathematics in Lean* continue where Chapters 2 and 31 stop. Every Mathlib section of Part IV is an entry point into a library with hundreds of thousands of theorems.
- **Type theory and semantics.** Pierce's *Types and Programming Languages* and Harper's *Practical Foundations for Programming Languages* cover the ground of Parts I–III in depth; *Software Foundations* and *Programming Language Foundations in Agda* mechanize it, as we did for our toy languages.
- **Linear logic and substructural types.** Girard's original paper, Walker's chapter on substructural type systems, and Pfenning's lecture notes take Part V further.
- **Rust's foundations.** The RustBelt papers, Ralf Jung's thesis, Stacked and Tree Borrows, and the work on Polonius are where the questions of Part VI are being answered today.
- **Verifying Rust.** Aeneas, Verus, Creusot, Prusti and Kani bring the methods of Chapter 31 to real Rust code. Aeneas, in particular, translates Rust into Lean, closing the loop between the two languages of this book.

33.11 Summary

- "Affine" means weakening without contraction: every assumption is used at most once. "Based on" can be historical, specificational or implementational.
- Ownership is affine in the specificational sense, and the usage theorem of Chapter 22 is the precise statement: rustc's E0382 is a failed context split. Three amendments: `Copy` is the exponential and restores contraction; `Clone` is an explicit duplication term; weakening runs a destructor, and nothing forces consumption, so Rust is affine, not linear. Rust refines the discipline to places.
- Borrowing is not affine. `&T` is fully structural; the conflicts the borrow checker reports are between different variables pointing to one place; affine typing with duplicable references is unsound. The repairs, lend-and-return and regions, are about time.
- In the borrow calculus of Chapter 28, the aliasing bug is a failed split of the owner, but only inside a scoped borrow, and the escape check is beyond what affinity explains. Lifetimes are region variables, their order is subtyping, variance is functoriality, and references are permissions.
- rustc checks moves and borrows with dataflow analyses on MIR, not by building derivations: the implementational sense is false for both halves.

- Affinity reappears one level down: Iris, the logic of RustBelt's soundness proof, is an affine separation logic.
- Say: Rust's ownership is affine; its borrowing is regions and permissions.

33.12 Exercises

1. For each of the errors E0382, E0499, E0502, E0505, E0597 and E0204, write a five-line program that triggers it, and say in one sentence which part of the theory rejects it: a failed context split, a side condition of the exponential, a permission conflict, or a region constraint. *Hint:* the error-code table above, and the compile-fail programs of Chapters 6, 23, 26 and 29, which you can adapt.
2. Imagine a Rust with an opt-in linear marker trait: values of a `Linear` type may not be dropped implicitly, only consumed by a function that takes them by value. Which structural rule does `Linear` remove? Rewrite the connection typestate of Chapter 25 with it, and explain which mistake it would now catch. Then explain why `mem::forget` and `Rc` would need special treatment. *Hint:* Chapter 24's discussion of what linearity would buy, and its Lean theorem about `forget` breaking "exactly once".
3. Take the dangling-reference term of Chapter 26 and type it in the calculus of Chapter 28. Which rule fails, and is the failure an affine one (a failed split) or a region one (the escape check)? *Hint:* consider which variables the borrow construct consumes, and what type the body of the borrow would need.
4. In Lean, write down the table "which structural rules Rust keeps" as a function from a small inductive type of "kinds of value" (owned, `Copy`, shared reference, mutable reference) to a structure of three booleans, and prove by `decide` that exactly two kinds lack contraction. *Hint:* `deriving DecidableEq` on the inductive type, and a theorem of the form `(kinds.filter (fun k => !(rules k).contraction)).length = 2`.
5. Write the "How to say it correctly" paragraph for three different audiences: a Rust beginner, a programmer who knows Haskell, and a logician who has never used Rust. What does each of them need you to define, and what can you leave out? *Hint:* the beginner needs moves and borrows, not structural rules; the logician needs `Copy` and `&T`, not lifetimes syntax.